

Universidad Rafael Landívar
Compiladores
Ing. Moises Antonio Alonso Gonzalez

AvengerScript Compiler

Documentación Técnica del Compilador

Nery Samuel Hernández Herrera— 1098824
Gabriel Emilio Toyom Jimenez — 1051524
Marc Wilhelm Schaub García — 1243424

1. Introducción y Objetivos

1.1 Introducción

AvengerScript IDE es un entorno de desarrollo integrado (IDE) web que implementa un compilador completo para AvengerScript, un lenguaje de programación de propósito educativo con temática del universo Marvel. El sistema permite escribir código AvengerScript directamente en el navegador, compilarlo a Java, visualizar el código generado y ejecutarlo de forma interactiva en tiempo real.

El proyecto nace como aplicación práctica de los fundamentos teóricos del curso de Compiladores, integrando todas las fases clásicas del proceso de compilación —análisis léxico, análisis sintáctico, análisis semántico y generación de código— en una solución funcional, visualmente atractiva y completamente operativa.

1.2 Objetivos

Objetivo General

Diseñar e implementar un compilador completo para el lenguaje AvengerScript que traduzca código fuente a Java válido y lo ejecute de forma interactiva desde un IDE web.

Objetivos Específicos

- Implementar el análisis léxico mediante ANTLR 4 para tokenizar el lenguaje AvengerScript.
- Implementar el análisis sintáctico generando un árbol de parse que refleje la estructura gramatical del programa.
- Implementar el análisis semántico validando tipos, declaraciones de variables, definición de funciones y correctitud de retornos.
- Implementar la generación de código traduciendo el árbol semántico a código Java válido y ejecutable.
- Integrar ejecución interactiva permitiendo entrada de datos del usuario en tiempo real.

2. Descripción del Lenguaje AvengerScript

2.1 Filosofía del Lenguaje

AvengerScript es un lenguaje imperativo, fuertemente tipado y de propósito educativo cuya sintaxis reemplaza las palabras clave convencionales por nombres de personajes y conceptos del universo Marvel. Esto facilita el aprendizaje al añadir una capa mnemónica temática sobre conceptos de programación estándar.

2.2 Tipos de Datos

Keyword AvengerScript	Tipo Java	Descripción	Personaje
stark	int	Número entero	Tony Stark (Iron Man)
banner	double	Número decimal	Bruce Banner (Hulk)
rogers	String	Cadena de texto	Steve Rogers (Cap. América)
thor	boolean	Valor booleano	Thor

2.3 Palabras Clave del Lenguaje

Keyword	Equivalente	Uso
jarvis	=	Asignación de valor
vision	if	Condicional
wanda	else	Alternativa
loki	while	Bucle while
fury	for	Bucle for
nebula	print	Impresión en consola
gamora	input	Lectura de entrada

2.4 Operadores

Operador	Descripción
+	Suma / concatenación de strings
-	Resta
*	Multiplicación
/	División
==	Igualdad
!=	Desigualdad
<	Menor que
>	Mayor que

2.5 Sintaxis del Lenguaje

Declaración y Asignación

```
stark x jarvis 5;
banner pi jarvis 3.14;
rogers nombre jarvis "Avenger";
thor activo jarvis TRUE;
```

Condición vision / wanda

```
vision (x > 0) {
    nebula("Positivo");
} wanda {
    nebula("Negativo o cero");
}
```

Bucle loki (while)

```
loki (x < 10) {
    nebula(x);
    x jarvis x + 1;
}
```

Bucle fury (for)

```
fury (stark i jarvis 0; i < 10; i jarvis i + 1) {
    nebula(i);
}
```

Funciones

```
stark sumar(stark a, stark b) {
    return a + b;
}
```

```
bob saludar(rogers nombre) {
    nebula("Hola, " + nombre);
}
```

3. Arquitectura del Sistema

3.1 Visión General

El sistema sigue una arquitectura de tres capas bien definidas: frontend web, backend compilador y proceso de ejecución Java.

Capas del Sistema

- **Capa de Presentación:** Monaco Editor + HTML/CSS/JS puro en el navegador
- **Capa de Backend:** Spring Boot 2.7 (Java 11) en puerto 8080, capaz de manejar REST y WebSocket
- **Capa de Compilación:** ANTLR 4.13 + SemanticVisitor + EvalVisitor
- **Capa de Ejecución:** Proceso Java hijo con streaming de I/O via WebSocket

3.2 Tecnologías Utilizadas

Capa	Tecnología	Versión	Rol
Backend	Spring Boot	2.7	Servidor HTTP y WebSocket
JDK	Java	11	Plataforma de ejecución
Compilador	ANTLR	4.13	Generación de lexer/parser
Frontend	Monaco Editor	CDN	Editor de código
Frontend	HTML/CSS/JS	Puro	Interfaz de usuario
Comunicación	WebSocket	—	Streaming de I/O en tiempo real
Generador	javac	11	Compilación del código generado

3.3 Flujo de Comunicación

REST API — Compilación

El cliente envía POST /api/compile con el código fuente. El CompileController delega al CompilerService que invoca ANTLR y los visitors. La respuesta incluye: código Java generado, lista de errores y tabla de tokens.

WebSocket — Ejecución Interactiva

Mensaje del cliente	Dirección	Significado
{ type:"run", code }	→ Servidor	Iniciar ejecución del programa
{ type:"stdin", data }	→ Servidor	Enviar entrada al proceso
{ type:"kill" }	→ Servidor	Terminar el proceso
{ type:"stdout", data }	← Servidor	Salida del programa
{ type:"stderr", data }	← Servidor	Errores en tiempo de ejecución
{ type:"exit", code }	← Servidor	Proceso terminado

4. Pipeline del Compilador

El compilador de AvengerScript implementa el pipeline clásico de compilación en cinco fases secuenciales. Si cualquier fase detecta errores, el proceso se detiene y no avanza a la siguiente.

4.1 Fase 1 — Análisis Léxico

Componente: AvengerLexer (generado por ANTLR desde Avenger.g4)

El lexer recorre el código fuente carácter por carácter y produce una secuencia de tokens. Cada token tiene un tipo, un valor lexema, número de línea y columna.

Categorías de Tokens

Categoría	Ejemplos
Palabras clave de tipos	stark, banner, rogers, thor, bob
Palabras clave de control	vision, wanda, loki, fury
Palabras clave de I/O	nebula, gamora
Operadores	jarvis, +, -, *, /, ==, !=, <, >
Delimitadores	(,), {, }, ;, ,,
Literales	42, 3.14, "hola", TRUE, FALSE
Identificadores	x, sumar, nombre
Comentarios / Espacios	//Hola Mundo

4.2 Fase 2 — Análisis Sintáctico

Componente: AvengerParser (generado por ANTLR desde Avenger.g4)

El parser toma el stream de tokens y construye el árbol de parse (Concrete Syntax Tree) verificando que la secuencia de tokens respeta la gramática. ANTLR genera un parser LL(*) con análisis predictivo y lookahead adaptativo.

4.3 Fase 3 — Análisis Semántico

Componente: SemanticVisitor.java

El SemanticVisitor recorre el árbol de parse y realiza validaciones que el análisis sintáctico no puede detectar. Implementa el patrón Visitor de ANTLR.

Validaciones Realizadas

Validación	Error generado
Variables declaradas antes de usar	Variable 'x' no declarada
Compatibilidad de tipos en asignación	Tipo incompatible: se esperaba stark, se encontró banner
Funciones definidas antes de llamar	Función 'sumar' no definida
Cantidad correcta de parámetros	Función 'sumar' espera 2 argumentos, se recibieron 3
Return en funciones con tipo	Función 'calcular' debe retornar un valor de tipo stark

4.4 Fase 4 — Generación de Código

Componente: EvalVisitor.java

El EvalVisitor recorre el árbol validado y construye el contenido del archivo Traduccion.java mediante un StringBuilder. Genera código Java estándar con imports, clase y método main.

Mapeo de Construcciones Avengerscript a Traducción Java

Avengerscript	Java generado
stark x jarvis 5;	int x = 5;
banner pi jarvis 3.14;	double pi = 3.14;
rogers s jarvis "hola";	String s = "hola";
thor b jarvis TRUE;	boolean b = true;
nebula(x);	System.out.println(x);
gamora(nombre);	nombre = scanner.nextLine();
vision (x>0) { } wanda { }	if (x > 0) { } else { }
loki (x<10) { }	while (x < 10) { }
fury (stark i jarvis 0; i<10; i jarvis i+1)	for (int i = 0; i < 10; i = i + 1)

4.5 Fase 5 — Compilación y Ejecución

Componentes: CompilerService.java, TerminalWebSocketHandler.java

Una vez generado el archivo Traduccion.java, se invoca javac con codificación UTF-8 para compilarlo. Si la compilación es exitosa, se lanza un proceso Java hijo cuyo stdout es streamado en tiempo real al browser via WebSocket. El proceso tiene un timeout de 30 segundos para prevenir ejecuciones infinitas.

5. Estructura del Proyecto

5.1 Responsabilidades de Componentes

Backend — Spring Boot

Archivo	Responsabilidad
IdeApplication.java	Punto de entrada. Inicia Spring Boot y abre el navegador en localhost:8080.
WebSocketConfig.java	Registra el endpoint WebSocket /ws/terminal y configura el handler.
CompileController.java	Controlador REST. Recibe POST /api/compile y retorna tokens, Java y errores.
TerminalWebSocketHandler.java	Maneja la sesión WebSocket. Gestiona mensajes run, stdin y kill.
CompilerService.java	Núcleo de la compilación. Invoca ANTLR, los visitors y el proceso javac.

Compilador — ANTLR y Visitors

Archivo	Responsabilidad
Avenger.g4	Define la gramática completa del lenguaje AvengerScript.
AvengerLexer.java	Generado automáticamente. Tokeniza el código fuente.
AvengerParser.java	Generado automáticamente. Construye el árbol de parseo.
SemanticVisitor.java	Recorre el árbol validando tipos, declaraciones y funciones.
EvalVisitor.java	Recorre el árbol generando el código Java en Traduccion.java.
Variable.java	Modelo de dato para representar una variable con nombre y tipo.

Frontend

Archivo	Responsabilidad
index.html	Estructura HTML del IDE: editores, paneles, tabs y menús.
app.js	Lógica de la aplicación: Monaco Editor, WebSocket, compilación, syntax highlighting.
style.css	Tema visual JARVIS HUD: colores oscuros con acentos rojo y dorado.

6. El IDE Web

6.1 Descripción General

El IDE web está implementado con HTML, CSS y JavaScript puro, sin frameworks frontend. Usa Monaco Editor vía CDN para ofrecer una experiencia de edición similar a Visual Studio Code.

6.2 Funcionalidades

Editor de Código Avengerscript

- Motor Monaco Editor
- Syntax highlighting personalizado para keywords de Avengerscript
- Autocompletado con las palabras clave del lenguaje
- Numeración de líneas y resaltado de la línea activa

Editor Java (Solo Lectura)

- Muestra el código Java generado por el compilador
- Syntax highlighting completo de Java mediante Monaco
- Se actualiza automáticamente tras cada compilación

Panel Inferior — Tres Tabs

- Tab Salida: Terminal interactivo en tiempo real, muestra stdout via WebSocket
- Tab Errores: Lista todos los errores con fase, línea, columna y mensaje descriptivo
- Tab Tokens: Tabla con tipo, lexema, línea y columna de cada token

Menú de Ejemplos Precargados

No.	Ejemplo
1	Hola Mundo
2	Aritmética
3	If/else
4	If anidado
5	Ciclo While
6	Ciclo For
7	Declaración de funciones
8	Declaración de funciones sin retorno (Void)
9	Entrada de texto desde teclado

Atajos de Teclado

Atajo	Acción
Ctrl + Enter	Compilar y ejecutar
Ctrl + Shift + B	Solo compilar

6.3 Tema Visual JARVIS HUD

Elemento	Color
Fondo principal	#0a0e1a (azul marino muy oscuro)
Acentos primarios	#c0392b (rojo Iron Man)
Acentos secundarios	#f39c12 (dorado)
Acentos terciarios	#3498db (azul arc reactor)
Texto principal	#e0e6f0 (blanco azulado)

7. Ejemplos de Programas

7.1 Hola Mundo

AvengerScript:

```
nebula("¡Hola, Avengers!");
```

Java generado:

```
public class Traduccion {  
    public static void main(String[] args) {  
        System.out.println("¡Hola, Avengers!");  
    }  
}
```

7.2 Variables y Tipos

AvengerScript:

```
stark edad jarvis 25;  
banner altura jarvis 1.82;  
rogers nombre jarvis "Tony Stark";  
thor esAvenger jarvis TRUE;  
nebula(nombre);  
nebula(edad);
```

Java generado:

```
public class Traduccion {  
    public static void main(String[] args) {  
        int edad = 25;  
        double altura = 1.82;  
        String nombre = "Tony Stark";  
        boolean esAvenger = true;  
        System.out.println(nombre);  
        System.out.println(edad);  
    }  
}
```

7.3 Factorial Recursivo

AvengerScript:

```
stark factorial(stark n) {  
    vision (n == 0) {  
        return 1;  
    }  
    return n * factorial(n - 1);  
}  
rogers entrada;  
gamora(entrada);  
stark num jarvis entrada;  
nebula("Factorial de " + num + " es: " + factorial(num));
```

Java generado:

```
import java.util.Scanner;  
public class Traduccion {  
    public static int factorial(int n) {  
        if (n == 0) { return 1; }  
        return n * factorial(n - 1);  
    }  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        String entrada;  
        entrada = scanner.nextLine();  
        int num = Integer.parseInt(entrada);  
        System.out.println("Factorial de " + num + " es: " + factorial(num));  
    }  
}
```

8. Manejo de Errores

8.1 Principio General

El compilador implementa un modelo de detención anticipada: si se detectan errores en una fase, el proceso se detiene inmediatamente y no avanza a la siguiente. Esto evita errores en cascada confusos para el usuario.

8.2 Errores Léxicos

Se generan cuando el lexer encuentra un carácter que no pertenece a ningún token válido.

Formato: *ERROR LÉXICO [línea X, col Y]: Símbolo no reconocido → 'z'*

Código AvengerScript	Error generado
stark x jarvis 5@;	ERROR LÉXICO [línea 1, col 13]: Símbolo no reconocido → '@'
stark x jarvis \$valor;	ERROR LÉXICO [línea 1, col 13]: Símbolo no reconocido → '\$'

8.3 Errores Sintácticos

Se generan cuando la secuencia de tokens no corresponde a ninguna regla gramatical válida.

Código AvengerScript	Error generado
vision x > 0 { nebula(x); }	ERROR SINTÁCTICO [línea 1, col 7]: Se esperaba '(' después de 'vision'
stark x jarvis;	ERROR SINTÁCTICO [línea 1, col 14]: Token inesperado → ';'.

8.4 Errores Semánticos

Se generan durante el análisis semántico cuando el código es sintácticamente válido, pero viola reglas de tipos o alcance.

Situación	Error generado
Usar variable no declarada	ERROR SEMÁNTICO: Variable 'resultado' no ha sido declarada
Asignar tipo incompatible	ERROR SEMÁNTICO: Tipo incompatible en 'x': se esperaba stark, se encontró banner
Lllamar función no definida	ERROR SEMÁNTICO: Función 'calcular' no está definida
Número incorrecto de args	ERROR SEMÁNTICO: Función 'sumar' espera 2 argumentos, se recibieron 1
Función sin return	ERROR SEMÁNTICO: Función 'obtener' debe retornar un valor de tipo stark

8.5 Errores en Tiempo de Ejecución

Situación	Comportamiento
Timeout (>30 segundos)	Proceso terminado forzosamente, mensaje de timeout al cliente
gamora() sin entrada disponible	NoSuchElementException capturada, mensaje en stderr
División por cero	Excepción Java estándar, stderr enviado via WebSocket
Error de javac	Mensaje de error de compilación Java enviado al tab Errores

9. Conclusiones

9.1 Logros del Proyecto

El proyecto AvengerScript IDE logró implementar exitosamente un compilador completo con todas las fases del pipeline clásico de compilación:

- Análisis léxico funcional mediante ANTLR 4, con reporte de errores preciso (línea y columna).
- Análisis sintáctico mediante parser LL(*) generado por ANTLR, con detección de errores gramaticales.
- Análisis semántico implementado manualmente cubriendo declaración de variables, compatibilidad de tipos y correctitud de retornos.
- Generación de código Java válido, compilable y ejecutable, con soporte completo para todos los tipos y estructuras del lenguaje.

9.2 Decisiones de Diseño Relevantes

- **Uso de ANTLR 4:** Simplificó enormemente las fases léxica y sintáctica, permitiendo concentrar el esfuerzo en el análisis semántico y la generación de código.
- **WebSocket para ejecución interactiva:** Clave para lograr ejecución en tiempo real, permitiendo flujos de stdin/stdout sin latencia.
- **Detención anticipada de errores:** Produce mensajes más claros y evita errores en cascada confusos para el usuario.

9.3 Aprendizajes

- La construcción de un compilador real refuerza la comprensión de autómatas, gramáticas formales, tipos y semántica.
- El patrón Visitor de ANTLR es poderoso para implementar múltiples pasadas sobre el mismo árbol con diferentes objetivos.
- La integración frontend-backend-proceso requiere diseño cuidadoso del flujo de comunicación con WebSocket.
- El manejo de errores es tan importante como la funcionalidad principal: un compilador útil da mensajes claros y precisos.

9.4 Posibles Mejoras Futuras

Mejora	Descripción
Arreglos y listas	Soporte para tipos compuestos en AvengerScript
Más operadores	Operadores lógicos &&, , ! y módulo %
Alcance de variables	Implementar alcance léxico con pila de tablas de símbolos
Optimizaciones	Constant folding y eliminación de código muerto antes de generar Java
Depurador	Ejecución paso a paso con visualización del estado de variables